Parte II: Con un ataque adversario es posible alterar la clasificacion que un algoritmo hace sobre una imagen: a) modificando gradualmente muchos de los píxeles de la imagen, por ejemplo mediante el algoritmo FGSM o b) utilizando pegatinas en ciertas regiones (parches) como sería el caso de la utilizacion del algoritmo RP2.

IIa) Analiza en detalle el codigo P1-Src1.ipynb para entenderlo y verificar la utilizacion del algoritmo FGSM. Explica si el codigo funciona tal y como se espera.
Primero se importa todo lo necesario, las librerías de pytorch, torchvision, numpy etc. Se declara la lista de epsilons con los que podremos probar, se guarda la dirección de los pesos del modelo ya pre entrenado, y se pasa a declarar la arquitectura de la red para ese modelo preentrenado (podríamos cambiar algo la arquitecutra, pero requeriría hacerle finetuning entonces para este caso no tenemos porqué modificar nada, la dejamos tal cual). Después se carga el dataset MNIST con el conjunto de test y se le aplica una normalización de media 0.1307 y desviación estándar 0.3081, supongo que es porque dicho modelo fue preentreando con un conjunto de datos que seguía estas propiedades estadísticas.
Después se carga la red, y ya SÍ se comienza con el ataque FGSM.
La definición del ataque es correcta, toma como entrada una iamgen (que ojo porque viene normalizada), un epsilon y el data_grad que será el gradiente respecto de la función de coste que hayamos definido para este ataque. Básicamente se coge este resultado de la función de coste y sel epalica sign() que se queda con un +1 o -1 según su signo, después se crea array con los pixeles de la imagen perturbada cogiendo la imagen original usada para calcuar esa función de coste y se le suma epsilon que es el tamaño de la perturbación * el signo de antes (+1 o -1). Ojo aquí, esta implemmentación de FGSM es un untargeted attack, no tenemos un objetivo y como target, simplemente se coge el propio output del modelo de ahora y se trata de alejarnos de este para empeorar el rendimiento del modelo, por lo que si nosotros quisieramos hacer que el modelo devuelva un target en específico, tendríamos que modifiocar la función para que en lugar de ser image + epsilon*sign(coste), sea -, ya que de esta forma nos “acercamos” en la dirección del gradiente con el paso máximo de epsilon que es nuestro presupuesto, y este gradiente al haber sido calculado con respecto a una función de coste que tiene un target y, lo que nosotros nos interesa es llegar a ese target y, lo que nos da es la dirección de máximo crecimiento del gradiente, y ese máximo crecimiento en ese caso sería aumentar el error, pero como nos conviene acercarnos, por eso restamos en lugar de sumar la perturbación.

Podemos ver que hay una función denorm que trata de coger un batch de tensores y quitarles esa normalización que se les aplicó en el dataloader, esto es, porque cuando nostoros calculamos este ataque FGSM, queremos aplicar el epsilon en el sentido de pixeles raw, no normalizados, porque así un 0.1 realmente elo máximo que vamos a perturbar al modificar la imagen. (hablamos en linf entonces 0.1 es que la modificaicon mas grande de cada pixel sera de +-0.1)

En cuanto al flujo, tenemos simplemente test() que lo deja bien claro, se va iterando por las muestras de test del dataset loader con el conjunto de test, se activa para que la imagen se le calucle el gradiente al hacer backpropagation con el autograd de pytorch, entonces asi se calcula la pérdida que es la NLL loss, osea la negative likelihood. Cuando nuestro modelo hace el output = model(data), ese output tiene que devolver un vector con 10 valores, que son de las 10 clases del dataset, y para cada valor lo que es, es básicamente una probabildad de cada output y_i para esa imagen x de entrada, entonces cuando hacemos el cálculo de esta función de pérdida, obtenemos por ejemplo que el target correcto es y = 2, porque el número de la imagen es un 2, entonces en loss = F.nll_loss(output,target), la función de pérdida coge ael valor dentro de output el que tiene y_i = 2 , por ejemplo 0.8 output → logp(y_i=2,x)=0.8, y lo pone negativo, -log(0.8) = 0.223. Ahora bien, si la pérdida es grande (la likelihood es pequeña), logp(y_i=2|x) = 0.01, tenemos -log(0.01)=4.605, la pérdida es grande.

Ahora con la pérdida clara, pues entendemos que ese gradiente que hemos activado para la entrada x (imagen) antes, que en el código está como data, se ha calculado el gradiente de ella respecto de la pérdida, entonces ya podemos hacer la denormalización a la imagen, y después aplicar el ataque FGSM, volver a normalizar, y meterlo por el modelo, consiguiendo la predicción final “perturbada”.

IIb) Puedes utilizar el repositorio correspondiente al artículo del algoritmo RP2 o generar tu propio codigo para generar parches que provoquen un ataque adversario a un conjunto de imagenes. Analiza experimentalmente la influencia del tamano del parche sobre la tasa de exito del ataque e identifica el menor tamano encontrado que produzca una clasificacion erronea de forma consistente.
Para tanto este IIb) como el IIc) vamos a crear un notebook donde experimentar (P1_src2.ipynb).

Primero que nada, tenemos que tener claro como funciona este algoritmo RP2, porque a diferencia del FGSM, el RP2 no perturba toda la imagen, sino que se centra en perturbar ciertas áreas de ella, llamadas parches (mediante máscaras), y además de esto, añade un término a la función objetivo de NPS y otro que hace EoT, pero dicen en el paper que como es complejo hacer esto de forma sintética, y hacerlo solo de forma sintética se queda “corto”, añaden variantes de la imagen de la señal al conjunto de entrenamiento para tener muestras x_i en X^v que es el conjunto de transformaciones de donde se samplean las muestras x_i sobre las que se hace este Expectation.

$$\underset{\delta}{\operatorname{argmin}}\ \lambda||M_x \cdot \delta||_p + NPS + \mathbb{E}_{x_i \sim X^V} J(f_\theta(x_i + T_i(M_x \cdot \delta)), y^*)$$

(Básicmente en este último término se quedan con la perturbación delta cuya media esperada calculada sobre la prueba con cada muestra x_i de dentro de las transformaciones X^v sobre la función de coste es menor.
Además añaden términos de regularización como que el temaño de la perdida sea acotado según x norma, por eso lo de Lp de ahí.

Ahora con todo esto más claro, me hago una pregunta, y es, ¿cómo crean estos parches? ¿Cómo comienza la perturbación?
Pues muy sencillo, se divide en dos fases, la primera mediante una optimización basándose en la norma l1, buscamos sparsity, tenemos esta norma l1 que trata de hacer el número de píxeles modificado menor, ya que estamos haciendo minimización, y queremos reducir la pérdida, y de esta forma descurbimos dónde colocar la máscara, y después de esto, optmizadmos el ruido dentro de la máscara ya fijada, estas son las 2 fases.

Fase 1, generan perturbaciones usando regularización L1 con una máscara que ya ocupa el área de la señal. Tras obtener resultados, se quedna con estos como las áreas o regiones más vulnerables del objeto. Sería algo así esta fase 1:

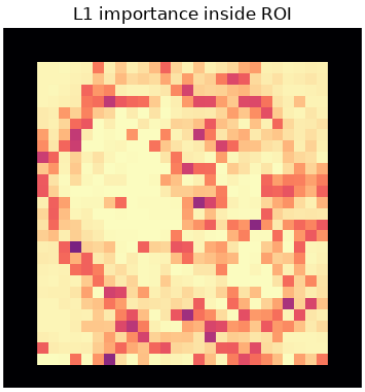
$$M\delta\|_1 + \mathbb{E}_{x_i \sim X_V}\left[J(f_\theta(x_i + T_i(M\delta)), y^*)\right]$$

Podemos observar que tenemos M cubriendo ya el área donde sí se va a aplicar la perturbación, osae si en un sitio si pues habrá un 1 y si no, un 0 etc etc.

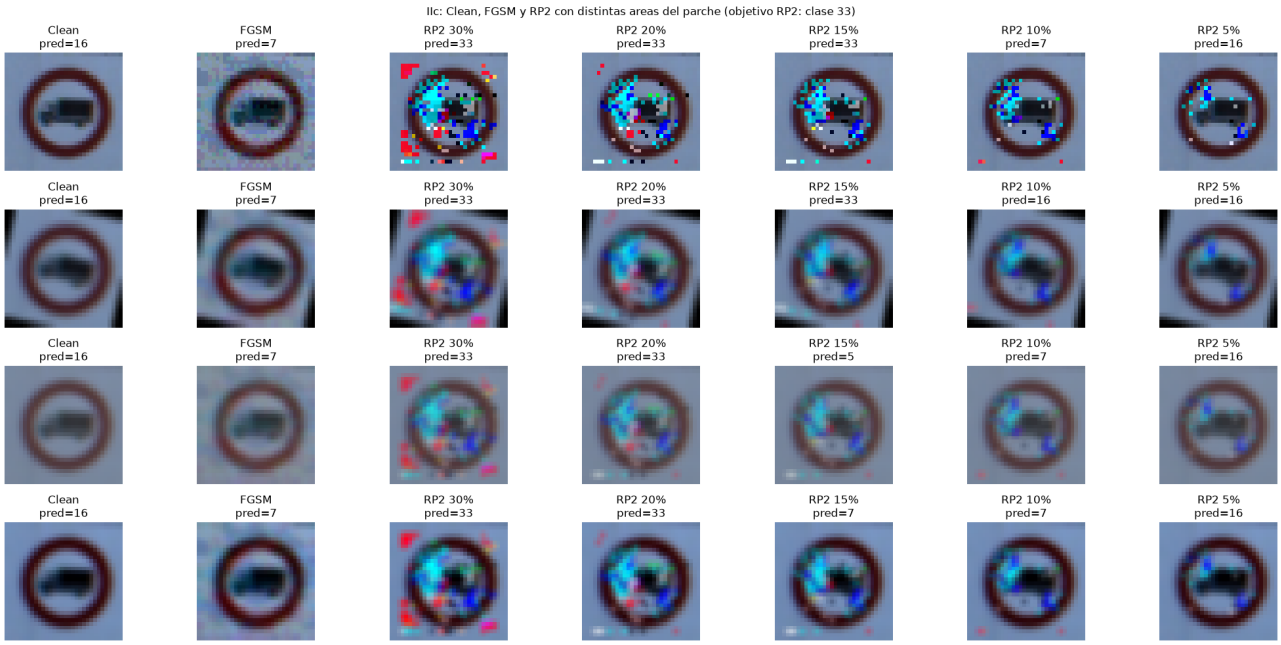
Fase 2, el paper dice que vuelven a calcular las perturbaciones usando regularización L2 con una máscara posicionada en estas “posiciones vulnerables” identificadas en el paso anterior. Entonces, básicamente colocan una máscara más pequeña sobre estas áreas y vueveln a optimizar, pero esta ver la optimización final tiene esta forma:

$$\underset{\delta}{\operatorname{argmin}}\ \lambda||M_x \cdot \delta||_p + NPS + \mathbb{E}_{x_i \sim X^V} J(f_\theta(x_i + T_i(M_x \cdot \delta)), y^*)$$

Ahora con todo esto claro, si podemos pasar al código y resultados. He creado un notebook llamado P1_src2.ipynb donde está todo.

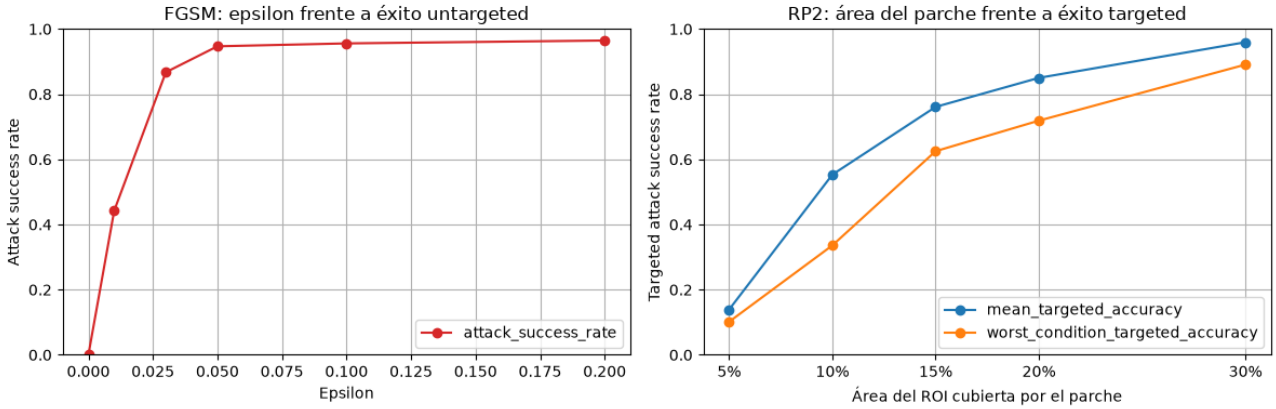


Aquí podemos ver cómo para la target class 33 sobre este dataset gtsrb los diferentes resultadosn tras atacar con diferentes tamaños de parches dentro las regiones vulnerables dentro del ROI de la imagen del objeto en la stage 1.



Podemos ver cómo al reducir el tamaño del parche (de la propia área de interés donde tenemos mayor “vulnerabilidad” nos quedamos con una parte menor en la stage 2 del algoritmo), el ASR baja y las predcciones dejan de ser la que nosotros queremos, incluso en ciertos puntos se convierte en la de FGSM.

IIc) Aplica ambos algoritmos a un conjunto de imagenes de senales de trafico, y analiza las diferencias entre su modo de aplicacion y sus resultados, realizando un ataque adversario y analizando la robustez del ataque frente a condiciones reales (tamano de la imagen de entrada, iluminacion, angulo de vision, etc.). Incluye al menos alguna grafica de (Ruido/distorsion/Epsilon) vs Resultado obtenido para caracterizar el funcionamiento de los algoritmos. Extrae conclusiones, volviendo a revisar las preguntas de la Parte I.



Podemos observar que para untargeted, FGSM funciona muy bien, y RP2 depende según el tamaño que escojamos tras sacar del stage 1 las áras más vulnerables, pues el ASR será mayor o menor. Eso sí, FGSM no podremos usarlo en un caso real ya que modifica toda la imagen, mientras que RP2 sí que podremos ya que se limita un área dentro del ROI.

IId) ¿Cómo podrías proteger tu sistema de IA frente a los dos tipos de ataque? En una práctica futura implementaremos mecanismos de protección frente a ataques adversarios pero por ahora lo analizamos sólo de manera teórica.

Aunque se hayan demostrado ante ataques “menos potentes” como FGSM que el distillation al entrenar el modelo funcione, para ataques más potentes como el de C&W no nos sirve, pues lo más eficaz diría que es el entrenamiento con muestras adversariales, donde el accuracy en clean tests será algo peor pero sobre las muestras adversariales no caerá.